Mohamed bin Zayed University of Artificial Intelligence

HTTP/REST Concepts

Designing Simple REST Endpoints

Building the foundation for scalable web services

HTTP vs REST: What's the Difference?

Mohamed bin Zayed University of Artificial Intelligence

HTTP - The Protocol

HyperText Transfer Protocol

HTTP is a communication protocol - the rules for how computers send and receive data over the web.

Think of it like the postal system: it defines how to address envelopes, what stamps to use, and how mail gets delivered.

HTTP provides:

Methods (GET, POST, PUT, DELETE)

Status codes (200, 404, 500)

Headers and body structure

REST - The Style

REpresentational State Transfer

REST is an architectural style - a set of design principles for building web APIs that are simple, scalable, and consistent.

Think of it like grammar rules: it defines how to structure your sentences (API calls) clearly.

REST provides:

Resource-based design (/users, /orders)

Stateless communication

Uniform interface patterns

Key insight: REST uses HTTP as its transport layer. HTTP is the "how" of communication; REST is the "what" and "why" of API design.

What Does REST Actually Mean?

Mohamed bin Zayed University of Artificial Intelligence

RE

Representational

Data is sent as a representation (like JSON) of the actual resource stored on the server.

S

State

The current condition of a resource. When you GET /users/123, you get the current state of that user.

T

Transfer

Moving that representation between client and server over HTTP.

EXAMPLE: Getting a user profile

```
GET /users/123
```

```
Response: { "id": 123, "name": "Alice", "email": "alice@example.com" }
```

This JSON is a representation of the state of user 123, transferred to your client.

Roy Fielding coined REST in his 2000 PhD dissertation. It describes how the web already worked and formalized it into design principles for APIs.

Why This Lesson Exists

The foundation for effective API development

Mohamed bin Zayed University of Artificial Intelligence

SHARED LANGUAGE

Client and server teams coordinate quickly with common vocabulary

PREDICTABLE APIs

Engineers build features without constant rework or surprises

STANDARD PROTOCOLS

HTTP avoids custom protocols that slow integration

EVOLVABILITY

REST concepts keep services adaptable as requirements change

CLEAR OWNERSHIP

Endpoint clarity makes debugging fast and responsibility clear

TOOL READINESS

Foundation needed before frameworks like FastAPI can be effective

Today's Journey

Mohamed bin Zayed University of Artificial Intelligence

1

HTTP Basics

Why it's the default

2

REST Constraints

Reduce complexity

3

Resources

Data modeling

4

Endpoint Design

Easy to use

5

Error Handling

Status codes

6

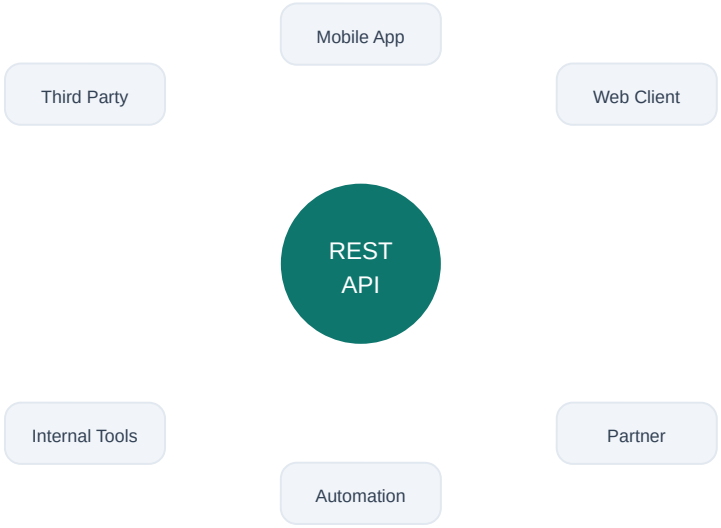
Future Ready

Testing, docs

> Design decisions today determine how fast we can iterate tomorrow

Why Web APIs Matter

Turning business capabilities into reusable services



Team Independence

Teams move independently instead of waiting on UI changes

Single Source of Truth

Reduce duplication by exposing consistent data across systems

Experimentation

Clients can be swapped, added, or modified without backend changes

Stability Contract

APIs preserve stability during rapid change and growth

Why HTTP Is the Backbone

Mohamed bin Zayed University of Artificial Intelligence

Universal

Any language or device can talk to your service. No special libraries needed.

Mature Tooling

Decades of tools for development, debugging, monitoring, and testing.

Observable

Text-based protocol makes issues easier to diagnose and debug.

Standard Ports

Port 80/443 work everywhere. Simpler deployment with no firewall headaches.

Built-in Caching

Native cache support essential for performance. CDNs work out of the box.

Stateless Scale

Stateless architecture makes horizontal scaling easy. Add servers anytime.

Key insight: HTTP's ubiquity means integration costs stay low as your ecosystem grows

Request-Response Model

Mohamed bin Zayed University of Artificial Intelligence

REQUEST

```
// Method + URL = Intent
GET /api/users/123

Headers
Authorization: Bearer token
Accept: application/json

Body (optional)
{ "name": "data" }
```

→

RESPONSE

```
// Status = Outcome
200 OK

Headers
Content-Type: application/json
Cache-Control: max-age=3600

Body
{ "id": 123, "name": "..." }
```

METHOD
Action intent

HEADERS
Metadata

BODY
Optional payload

STATUS
Machine-readable

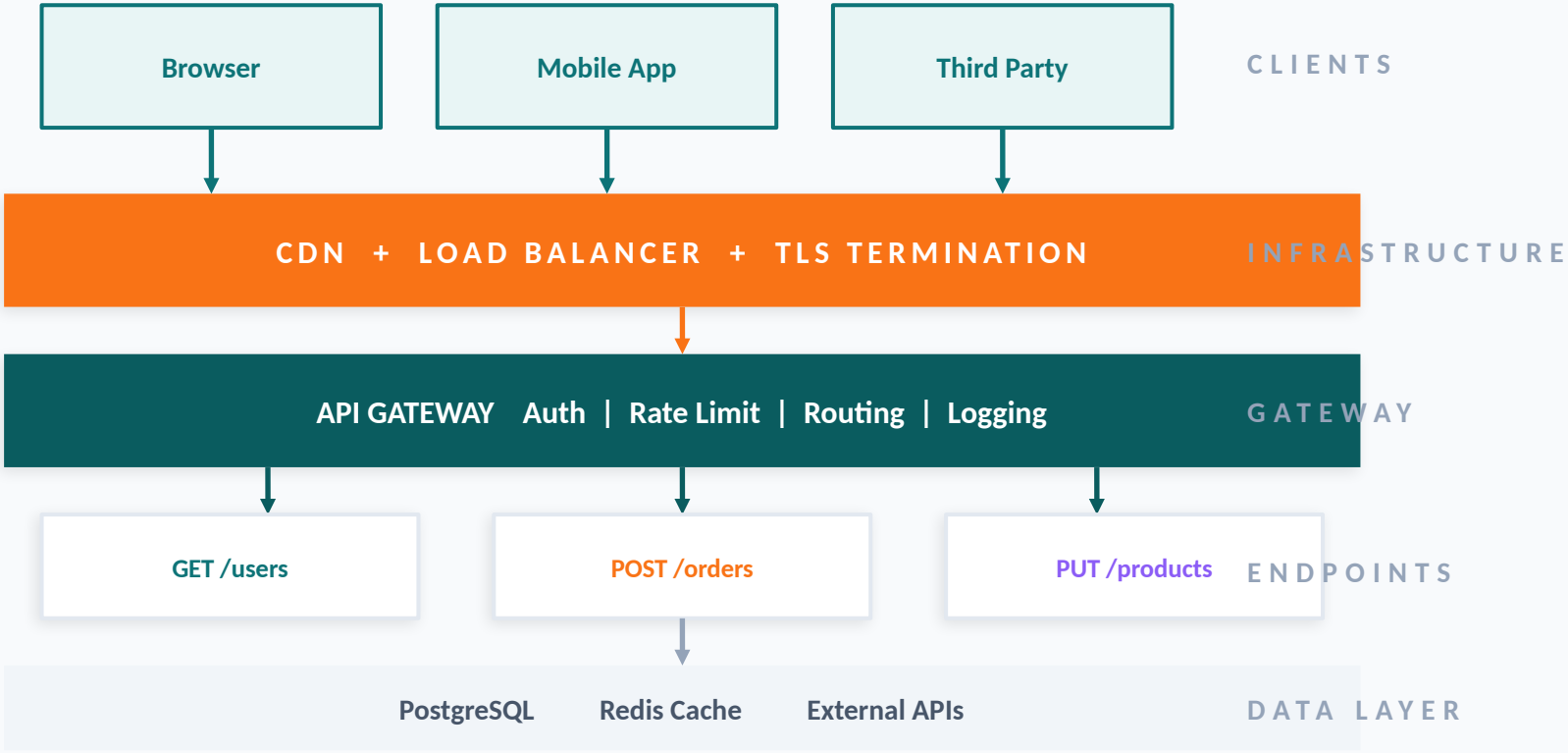
HEADERS
Cache hints

BODY
Data payload

Why it matters: Explicit intent + explicit outcome = reliable automation, easy retries, and predictable caching

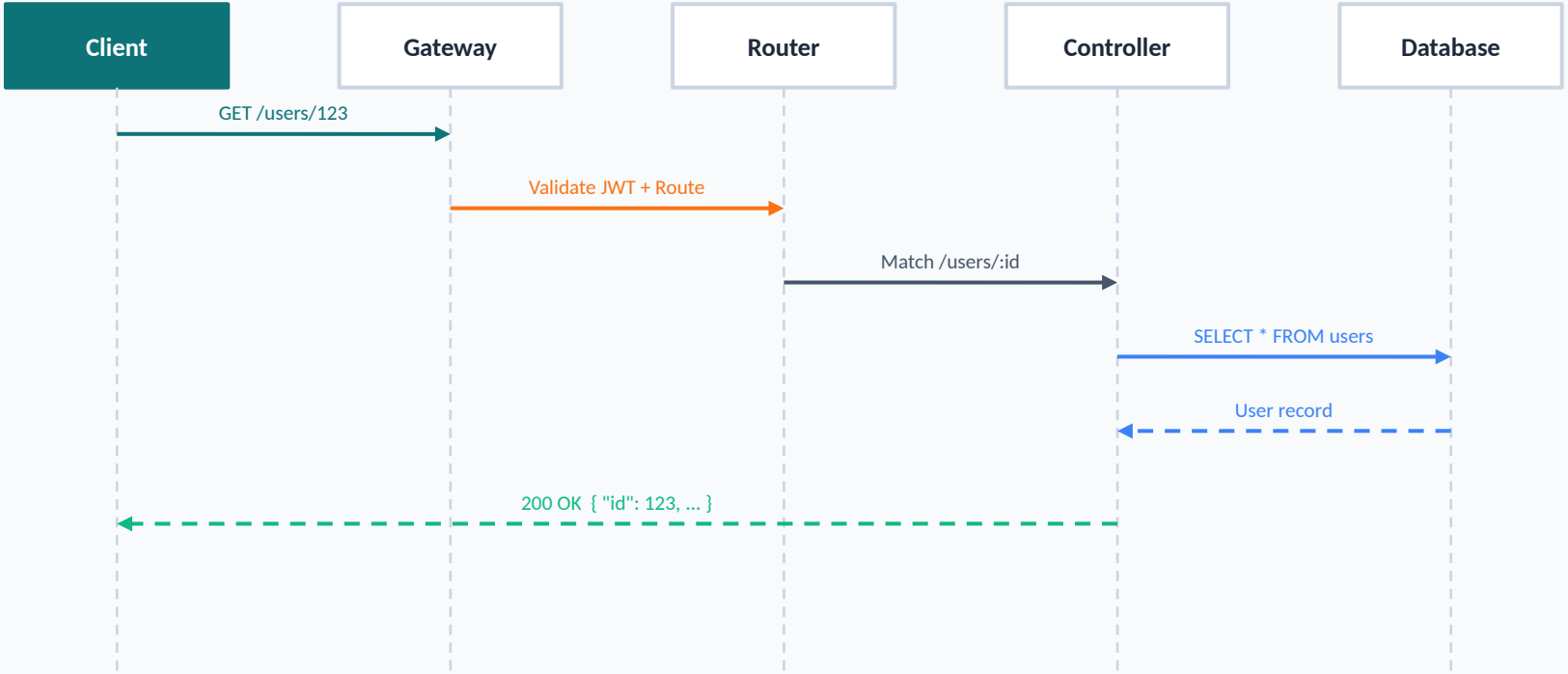
REST API Architecture

How layers work together in a production system



Request Lifecycle

What happens when a client sends GET /users/123



Each layer has a single responsibility. This makes testing, debugging, and scaling independent.

HTTP Methods and Intent

Using the right method enables tooling, proxies, and caches to help you

Mohamed bin Zayed University of Artificial Intelligence

GET

Read / Retrieve

Safe and cacheable. Protects data from accidental changes.

SAFE CACHEABLE

POST

Create

Creates new resources. Clarifies when state changes expected.

NOT SAFE NOT IDEMPOTENT

PUT

Replace

Full replacement. Supports complete updates and consistency.

IDEMPOTENT

PATCH

Partial Update

Handles partial updates. Reduces bandwidth and complexity.

PARTIAL

DELETE

Remove

Clear lifecycle control for data you no longer need.

IDEMPOTENT

Key Concepts

Safe: No server state changes

Idempotent: Same result if repeated

Cacheable: Response can be stored

Status Codes as Communication

Consistent codes prevent clients from hardcoding fragile heuristics

Mohamed bin Zayed University of Artificial Intelligence

2xx
SUCCESS

200 OK
Standard success

201 Created
New resource made

204 No Content
Success, no body

Client proceeds

4xx
CLIENT ERROR

400 Bad Request
Invalid syntax

401 Unauthorized
Auth required

404 Not Found
Resource missing

Client must fix

5xx
SERVER ERROR

500 Internal Error
Server failure

502 Bad Gateway
Upstream failed

503 Unavailable
Temp overload

Client can retry

The Six REST Constraints

Trading short-term shortcuts for long-term stability and scalability

Mohamed bin Zayed University of Artificial Intelligence

1. Client-Server

Separation of concerns enables independent evolution of frontend and backenc

2. Stateless

Each request contains all context needed. Simplifies servers and enables horizontal scalinc

3. Cacheable

Responses must define if cacheable. Reduces load and cuts infrastructure costs

4. Uniform Interface

Consistent API patterns across all endpoints. Enables tooling and reduces learning curve

5. Layered System

Clients cannot tell if connected directly or through intermediary (load balancer, cache, gateway)

6. Code on Demand (Optional)

Servers can extend client functionality by sending executable code (rarely used)

The Trade-off: Accept constraints today to avoid redesign costs tomorrow. These constraints guide design so systems scale without major rewrites.

Stateless in Practice

Each request must carry everything the server needs

STATEFUL (avoid)

```
# Server stores session — ties client to one server
@app.post("/login")
def login(creds: Credentials):
    user = authenticate(creds)
    session["user_id"] = user.id  # Stored on THIS server
    return {"msg": "logged in"}

@app.get("/orders") # Which server am I on??
def get_orders():
    user_id = session["user_id"] # Breaks if routed elsewhere
```

STATELESS (correct)

```
# Token carries identity — any server can handle it
@app.get("/orders")
def get_orders(
    user: User = Depends(get_current_user)
):
    # user extracted from JWT in Authorization header
    # No server-side session needed
    return db.query(Order).filter(
        Order.user_id == user.id
    ).all()
```

What about heavy resources? (images, files, large datasets)

```
# Don't embed heavy data in the response — return URLs. Let CDNs and caches do the work.
GET /products/42
{
  "id": 42, "name": "Widget",
  "image_url": "https://cdn.example.com/products/42.jpg",   # CDN-cached, not in API response
  "manual_pdf": "https://cdn.example.com/docs/widget.pdf",  # Client fetches separately
  "dataset_url": "/api/v1/products/42/data?format=csv"     # Paginated or streamed
}
```

Resources and URIs

Stable URIs are critical for linking, caching, and long-term stability

URI STRUCTURE

https://api.example.com /v1 /users /123 /orders

BASE VERSION COLLECTION ID SUB-RESOURCE

Map to Domain Concepts

Resources reflect business entities

Hierarchy Shows Relationships

/users/123/orders = user's orders

Use Nouns, Not Verbs

✓ /users ✗ /getUsers

Predictable Paths

Reduce docs burden and onboarding time

Use Plurals for Collections

✓ /orders ✗ /order

Avoid Leaking Internals

✗ /tbl_user_records

Error Handling Done Right

Standard error shapes allow uniform handling and reduce support overhead

Mohamed bin Zayed University of Artificial Intelligence

RFC 7807 PROBLEM DETAILS

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json

{
"type": "/errors/validation",
"title": "Validation Failed",
"status": 422,
"detail": "Request has 2 errors",
"errors": [
{ "field": "email", "msg": "invalid" },
{ "field": "age", "msg": "required" }
]
}
```

type

URI reference for error documentation

title + detail

Human-readable summary and specifics

status

Matches HTTP status for logging

errors (extension)

Field-level details help users fix input

Why Standard Shapes?

Clients handle errors uniformly. No fragile string parsing.

Security Essentials

Security by design is cheaper than incident response later

Mohamed bin Zayed University of Artificial Intelligence

MANDATORY

- [ ] **TLS Everywhere**
  HTTPS prevents credential leakage in transit
- [ ] **Authentication**
  Token-based auth (JWT, OAuth) scales better
- **Authorization**
  Least privilege reduces blast radius
- [ ] **Input Validation**
  Blocks injection attacks

IMPORTANT

- **Rate Limiting**
  Prevents abuse and protects availability
- [ ] **Audit Logging**
  Diagnose incidents and meet compliance
- [ ] **CORS Configuration**
  Prevents unintended cross-site access
- [ ] **Error Sanitization**
  Don't leak stack traces in responses

Query Parameters for Flexibility

Mohamed bin Zayed University of Artificial Intelligence

EXAMPLE REQUEST

```
GET /orders?status=pending&sort=created&page=2&limit=20
```

Filtering

`?status=pending`

Return only matching items, reduce payload size

Sorting

`?sort=created`

Offload sorting to server, keep UI responsive

Pagination

`?page=2&limit=20`

Protect servers from unbounded responses

Field Selection

`?fields=id,name`

Return only needed fields, save bandwidth

Key benefit: Query params preserve resource identity while varying the view

REST CRUD in FastAPI

Methods, status codes, and validation in real Python code

```
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel

app = FastAPI()

class UserCreate(BaseModel):            # Pydantic validates input automatically
    name: str                       # Missing field → 422 Unprocessable Entity
    email: str

# GET /users — List collection
@app.get("/users", status_code=200)        # 200 OK
def list_users(skip: int = 0, limit: int = 20):  # Query params for pagination
    return db.get_users(skip=skip, limit=limit)

# GET /users/{id} — Single resource
@app.get("/users/{user_id}")
def get_user(user_id: int):
    user = db.get_user(user_id)
    if not user:
        raise HTTPException(status_code=404, detail="User not found") # 404
    return user

# POST /users — Create new resource
@app.post("/users", status_code=201)        # 201 Created
def create_user(user: UserCreate):          # Body validated by Pydantic
    return db.create_user(user)

# PUT /users/{id} — Full replacement
@app.put("/users/{user_id}")
def replace_user(user_id: int, user: UserCreate):
    return db.update_user(user_id, user)

# DELETE /users/{id} — Remove resource
```

Error Handling in Python

RFC 7807 Problem Details with FastAPI

```python
from fastapi.responses import JSONResponse
from fastapi.exceptions import RequestValidationError

# Custom exception handler for RFC 7807
@app.exception_handler(HTTPException)
def http_exception_handler(request, exc):
    return JSONResponse(
        status_code=exc.status_code,
        content={
            "type": f"/errors/{exc.status_code}",
            "title": http_status_phrase(exc.status_code),
            "status": exc.status_code,
            "detail": exc.detail,
        },
        media_type="application/problem+json"
    )

# Validation errors → field-level details
@app.exception_handler(RequestValidationError)
def validation_handler(request, exc):
    return JSONResponse(
        status_code=422,
        content={
            "type": "/errors/validation",
            "title": "Validation Failed",
            "status": 422,
            "detail": f"{len(exc.errors())} error(s)",
            "errors": [
                {"field": e["loc"][-1], "msg": e["msg"]}
                for e in exc.errors()
            ]
        }
    )
```

CLIENT RECEIVES

404 Not Found

```
{
  "type": "/errors/404",
  "title": "Not Found",
  "status": 404,
  "detail": "User not found"
}
```

422 Validation Error

```
{
  "type": "/errors/validation",
  "title": "Validation Failed",
  "status": 422,
  "detail": "2 error(s)",
  "errors": [
    {"field":"email","msg":"invalid"},
    {"field":"age","msg":"required"}
  ]
}
```

HTTPS: How TLS Works

The handshake that protects every API call

1. **Client Hello**
   Client sends supported TLS versions and cipher suites
2. **Server Hello + Certificate**
   Server picks cipher, sends its SSL certificate (public key)
3. **Certificate Verification**
   Client checks certificate against trusted Certificate Authorities
4. **Key Exchange**
   Client + server agree on a shared session key (asymmetric → symmetric)
5. **Encrypted Communication**
   All data encrypted with fast symmetric encryption (AES-256)

WITHOUT TLS — Anyone can see:

```
GET /api/users/123
Authorization: Bearer eyJhbGc... ← Token VISIBLE

Response: {"ssn": "123-45-6789"} ← Data EXPOSED
```

WITH TLS — Encrypted in transit:

```
\x17\x03\x03\x00\x1a\x8f\x92\xa1... ← Encrypted
\x17\x03\x03\x00\x2b\xc4\x7e\xd3... ← Encrypted

Attacker sees NOTHING readable
```

In practice: TLS terminates at your reverse proxy (nginx, Traefik) or cloud load balancer, not in Python. FastAPI runs behind it and receives plain HTTP internally.

JWT Authentication

Stateless tokens that prove identity

JWT STRUCTURE

HEADER — Algorithm + type

```
{"alg":"HS256","typ":"JWT"}
```

PAYLOAD — Claims (who, what, when)

```
{"sub":"user123","role":"admin","exp":1700000}
```

SIGNATURE — Tamper protection

```
HMACSHA256(header + "." + payload, secret)
```

FASTAPI IMPLEMENTATION

```
from jose import jwt
from fastapi import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer

SECRET = "your-secret-key"
oauth2 = OAuth2PasswordBearer(tokenUrl="login")

# --- Create token on login ---
def create_token(user_id: str, role: str):
    payload = {
        "sub": user_id,
        "role": role,
        "exp": datetime.utcnow() + timedelta(hours=1)
    }
    return jwt.encode(payload, SECRET, "HS256")

# --- Verify token on every request ---
def get_current_user(
    token: str = Depends(oauth2)
):
    try:
        payload = jwt.decode(token, SECRET, ["HS256"])
        return payload  # {"sub": "user123", "role": "admin"}
    except jwt.JWTError:
        raise HTTPException(status_code=401)

# --- Use in any endpoint ---
@app.get("/orders")
def list_orders(user=Depends(get_current_user)):
    return db.get_orders(user["sub"])
```

RBAC: Role-Based Access Control

Authorization decides what authenticated users can do

THE CONCEPT

	Create	Read	Update	Delete
admin	✓	✓	✓	✓
manager	✓	✓	✓	✗
viewer	✗	✓	✗	✗

401 Unauthorized

"Who are you?
Please log in."

403 Forbidden

"I know who you are,
but you can't do this."

FASTAPI IMPLEMENTATION

```
# --- Reusable role checker ---
def require_role(*allowed_roles):
    def checker(
        user = Depends(get_current_user)
    ):
        if user["role"] not in allowed_roles:
            raise HTTPException(
                status_code = 403,
                detail = "Insufficient permissions"
            )
        return user
    return checker

# --- Apply to endpoints ---
@app.get("/orders")
def list_orders(
    user = Depends(require_role("admin","manager","viewer"))
):
    return db.get_orders()

@app.delete("/orders/{order_id}")
def delete_order(
    order_id: int,
    user = Depends(require_role("admin")) # Only admins
):
```

Depends() is FastAPI's superpower: define security once, reuse everywhere. No decorators, no boilerplate.

API Versioning Strategy

Explicit versions reduce fear of evolution and encourage improvement

Mohamed bin Zayed University of Artificial Intelligence

URL Path

`/v1/users`

Most visible and intuitive. Easy to test.

RECOMMENDED

Header

`API-Version: 2`

Cleaner URLs. More RESTful purists prefer this.

ALTERNATIVE

Query Param

`?version=1`

Easy to add. Can break caching.

USE WITH CARE

Version Lifecycle

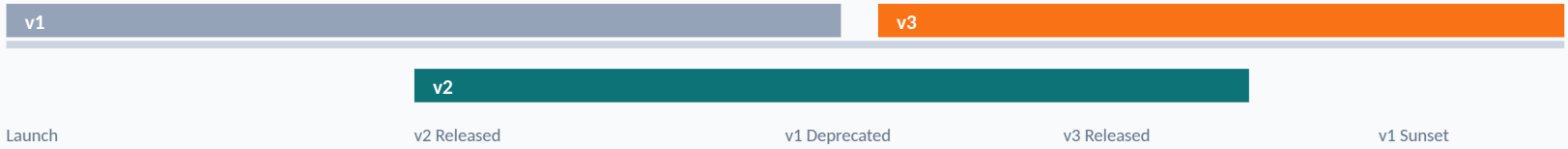
ACTIVE → **DEPRECATED** → **SUNSET**

v2 - Current | v1 - 6mo notice | v0 - Removed

Versioning Strategy Deep Dive

Managing API evolution without breaking clients

VERSION TIMELINE



URL Path (Recommended)

```
/v1/users
/v2/users
```

Pros: Visible, testable, cacheable

Cons: URL changes between versions

Custom Header

```
API-Version: 2
Accept: application/vnd.api+ json
```

Pros: Clean URLs, content negotiation

Cons: Harder to test, less discoverable

Query Parameter

```
/users?version= 2
/users?v= 2
```

Pros: Easy to add, no URL change

Cons: Breaks caching, clutters URLs

Security Architecture

Defense in depth: multiple layers of protection

SECURITY LAYERS

TLS / HTTPS	Encrypt all traffic
Rate Limiting	Prevent abuse
Authentication (JWT)	Verify identity
Authorization (RBAC)	Check permissions
Input Validation	Sanitize data
Your API Logic	Business code

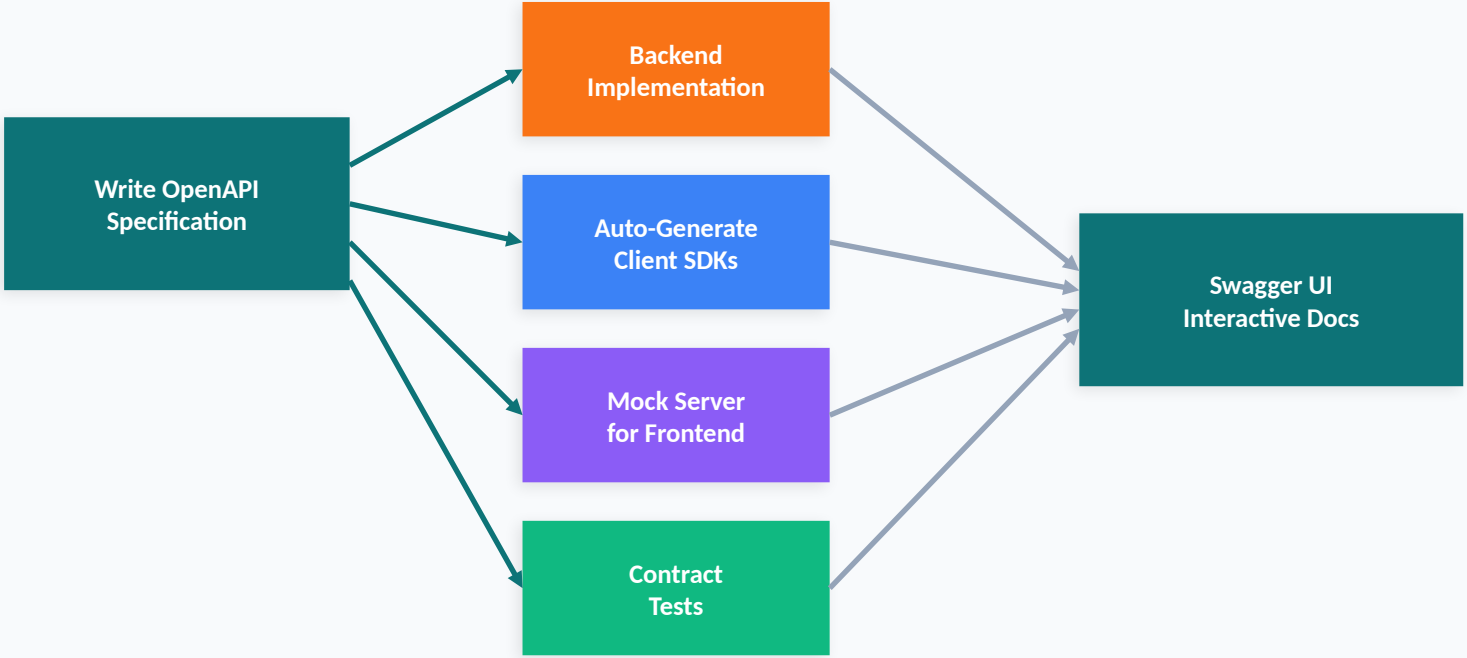
JWT AUTH FLOW

1. Client sends credentials
2. Server validates + issues JWT
3. Client stores token
4. Sends token in Authorization header
5. Server verifies signature
6. Grants access to resource

Defense in depth: each layer is independent, fails safely, and can be upgraded without affecting others.

API-First Development Workflow

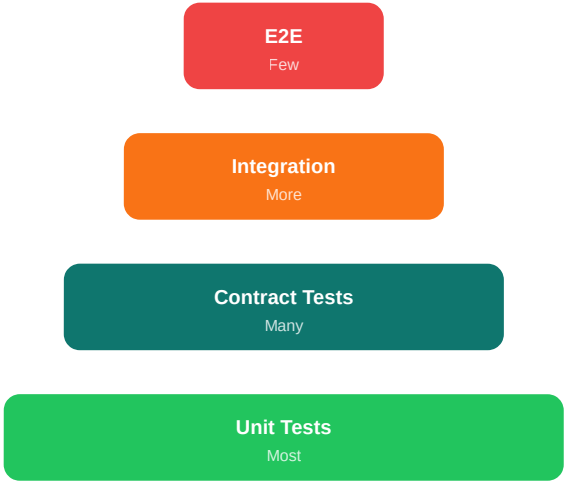
Design the contract before writing code



1. **Parallel Dev**
   Frontend + Backend work simultaneously
2. **Catch Issues Early**
   Design reviewed before implementation
3. **Single Source**
   One spec drives all artifacts
4. **Always Current**
   Docs auto-update from spec

Testing REST Endpoints

Testing protects API reliability, which protects your team's time



Test Pyramid for APIs

Unit Tests

Test business logic in isolation. Fast feedback, easy to maintain.

Contract Tests

Ensure client and server interpret fields the same. Catch breaking changes.

Integration Tests

Validate real HTTP behavior and middleware effects.

E2E / Load Tests

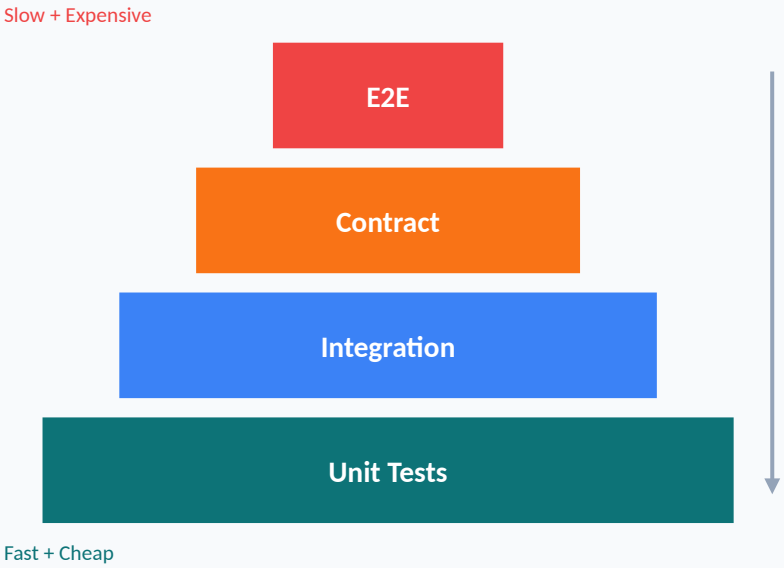
Full system validation. Performance under pressure. Use sparingly.

Don't Forget Error Paths

Test 4xx and 5xx scenarios. Fragile clients often fail on unexpected errors.

Testing Architecture

A layered strategy for API reliability



Unit Tests (70-80%)

pytest, unittest

Validation logic, data transforms, business rules, error formatting

Integration Tests (15-20%)

TestClient, httpx

HTTP routes, middleware chains, serialization, database queries, auth flow

Contract Tests (5-10%)

schemathesis, dredd

Response schema vs OpenAPI spec, breaking change detection, field types

E2E / Load Tests (<5%)

locust, k6, Postman

Full user journeys, performance benchmarks, spike testing, failure recovery

Don't forget error paths! Test 400, 401, 404, 422, 500 responses. Fragile clients fail on unexpected errors.

Anti-Patterns vs Best Practices

Side-by-side: what to avoid and what to embrace

ANTI-PATTERN		BEST PRACTICE	
`POST /getUserOrders`	→	`GET /users/123/orders`	*Let methods express intent*
`GET /deleteUser?id=1`	→	`DELETE /users/1`	*Respect method safety*
`200 { "error": true }`	→	`422 + Problem Details`	*Use status codes properly*
`/a/1/b/2/c/3/d/4`	→	`/items/4?context=a,b,c`	*Flatten deep hierarchies*
`{ "err": "something" }`	→	`RFC 7807 Problem Details`	*Standardize error shapes*
`/users (no version)`	→	`/v1/users`	*Always version from day one*

Pattern: shortcuts create technical debt. The correct approach pays dividends in maintainability and developer experience.

Mohamed bin Zayed University of Artificial Intelligence

REST API Best Practices

A checklist for production-ready APIs

Endpoint Design	HTTP Semantics	Security	Operations
✓ Use nouns, not verbs	✓ Correct method per operation	✓ TLS/HTTPS everywhere	✓ RFC 7807 error format
✓ Plural collection names	✓ Proper status codes (not just 200)	✓ JWT or OAuth2 tokens	✓ Paginate all list endpoints
✓ Version in URL path	✓ Cache-Control headers	✓ Validate all input fields	✓ Structured request logging
✓ Max 2 levels of nesting	✓ Idempotent PUT and DELETE	✓ Rate limit all endpoints	✓ OpenAPI specification
✓ Consistent naming across services	✓ Content-Type negotiation	✓ CORS configuration	✓ Health check endpoint
✓ Hide implementation details	✓ HATEOAS links (optional)	✓ Sanitize error responses	✓ Graceful deprecation notices

Richardson Maturity Model

The four levels of REST API maturity

TARGET FOR MOST APIs

Level 0

Swamp of POX

One URL, one method
POST /api for everything

→

Level 1

Resources

Multiple URLs
POST /users, POST /orders

→

Level 2

HTTP Verbs

Correct methods + status
GET /users, POST /users

→

Level 3

Hypermedia

Links in responses
"_links": { "self": "..." }

```
Level3: GET /users/123 → { "id":123, "_links": { "self":"/users/123", "orders":"/users/123/orders" } }
```

Practice and Next Steps

Mohamed bin Zayed University of Artificial Intelligence

1. Sketch endpoints for a simple domain
2. Write sample requests and responses
3. Test with curl or Postman
4. Apply naming and status rules
5. **Map designs into FastAPI next session**

PRACTICE DOMAIN IDEAS

- Bookstore: books, authors, orders
- Event ticketing: events, venues, tickets
- Todo app: lists, tasks, users
- Restaurant: menu, orders, reviews

Pick one and design: collection endpoints, single resource endpoints, query params, error responses

Practice now saves hours of debugging when real clients connect